Diseño y ejecución de un programa de Chaos Engineering para el fortalecimiento de la resiliencia en arquitecturas Cloud-Native sobre GKE

Informe técnico de Game Day: formulación de hipótesis, observabilidad en tiempo real, análisis empírico de fallos y remediación de debilidades sistémicas

Carlos Alberto Zambrano Braydi
0000395349

Henry Julián Salazar Salcedo
0000396117

Wolfran Alirio Pinzón Murillo
0000393439

Maestría en Arquitectura de Software, Universidad de La Sabana
Observabilidad en ambientes productivos — Unidad 3 — Grupo 11
Mg. María Fernanda Ochoa Paipilla
Octubre de 2026

Introducción y Marco Metodológico de Chaos Engineering

En los sistemas distribuidos modernos basados en microservicios y contenedores, asumir que los componentes individuales funcionarán de manera infalible es una premisa riesgosa. La red subyacente es inherentemente no determinista, las cargas de trabajo sufren contención de recursos y los fallos transitorios en cascada son inevitables a gran escala (Sachto y Walcer, 2022). Frente a esto, la disciplina de Site Reliability Engineering (SRE) plantea el principio de anti-fragilidad: los sistemas deben ganar robustez y resiliencia exponiéndose de forma deliberada y controlada a condiciones adversas antes de que se manifiesten en producción (Beyer et al., 2018).

El presente informe técnico documenta la solución a la actividad práctica de Chaos Engineering de la asignatura Observabilidad en Ambientes Productivos. El encargo consistió en diseñar y ejecutar un programa integral de caos sobre el clúster gestionado en Google Kubernetes Engine (GKE) implementado en el laboratorio anterior. Siguiendo la metodología de Principios del Caos (Basiri et al., 2016), se definió formalmente el estado estable (steady state), se formularon tres hipótesis de fallo basadas en métricas observables, se diseñaron tres experimentos con radio de impacto controlado y rollback garantizado, y se ejecutaron empíricamente dos pruebas clave: la inyección de latencia de red en el kernel mediante Chaos Mesh y la simulación de caída de instancia (Pod Failure) para validar la conmutación transparente en alta disponibilidad. Todo el proceso fue monitorizado en tiempo real a través del pipeline de observabilidad OTel, Prometheus y Jaeger, calculando métricas reales de detección (MTTD) y recuperación (MTTR).

Arquitectura Objetivo y Flujo Transaccional

La topología analizada responde al patrón de orquestación de servicios distribuido en el clúster otel-lab de GKE (proyecto otellabs, región us-central1):

1. service-a (Orquestador HTTP y API Gateway): Expone el endpoint público GET /order/{order_id} mediante un Service tipo LoadBalancer en la IP 34.57.219.46 (puerto 8000). Al recibir una solicitud externa, inicia el trace raíz distribuido, consulta la base de datos PostgreSQL local para obtener los datos de la orden y realiza una petición HTTP asíncrona hacia service-b utilizando httpx.AsyncClient con propagación de contexto W3C.

2. service-b (Servicio de Inventario): Servicio interno expuesto como ClusterIP (service-b-svc:8001). Expone el endpoint GET /inventory/{product_id}, continúa la traza activa recuperando el traceparent inyectado y consulta la tabla inventory en la base de datos.

3. PostgreSQL 16 (Almacenamiento Persistente): Instancia PostgreSQL 16 desplegada como StatefulSet con almacenamiento persistente (postgres-svc:5432). Para preservar la integridad transaccional de las tablas orders e inventory durante las inyecciones de caos, este recurso está formalmente etiquetado con chaos.exclude: "true", aislándolo de fallos destructivos a nivel de disco.

4. Pipeline de Observabilidad: Agente DaemonSet OpenTelemetry Collector Contrib (puerto 4317 gRPC / 4318 HTTP), Prometheus v2.52.0 (puerto 9090) con reglas de alerta AIOps y Jaeger UI v1.58 (puerto 16686). Cada pod incorpora el SDK oficial de Python para exportar métricas de solicitud, spans de negocio y logs estructurados.

Estado Estable y Formulación de las Tres Hipótesis de Fallo

Un principio fundamental del Chaos Engineering es que cualquier prueba debe contrastarse contra una definición cuantitativa de normalidad. Tras ejecutar una corrida de calibración de 50 peticiones concurrentes sobre el orquestador, se consolidó la siguiente línea base (steady state):

- Disponibilidad: 100% de respuestas con código HTTP 200 OK.
- Latencia: percentil 99 (p99) de 0.425 s y media de 0.282 s.
- Tasa de errores de servidor (5xx): 0.00%.
- Consumo de recursos: CPU inferior a 15 mCPU por pod en reposo relativo.

Con base en este comportamiento observable, se redactaron las tres hipótesis de fallo en formato canónico:

- **Hipótesis 1 (Latencia de Red — Dependencia Downstream):** «En estado estable, cuando inyectamos un retardo de red de 200 ms (+/-10 ms de jitter) en los pods de service-b mediante emulación tc qdisc en el kernel, esperamos que service-a mantenga la tasa de errores de servidor en 0%, que el cliente HTTPX no aborte por timeout y que la latencia p99 del sistema aumente de forma predecible sin superar el umbral de 5.0 s».

- **Hipótesis 2 (Pod Failure — Alta Disponibilidad N-1):** «En estado estable, cuando ocurre la terminación no programada (pod kill) de una de las dos réplicas de service-b, esperamos que el controlador de endpoints de Kubernetes redirija inmediatamente el tráfico al pod superviviente (N-1) sin que service-a perciba caídas y que el ReplicaSet reprograme la réplica faltante en menos de 10 segundos».

- **Hipótesis 3 (Agotamiento de Recursos — CPU Stress):** «En estado estable, cuando sometemos al 50% de las réplicas de service-a a un consumo sostenido del 80% de CPU mediante StressChaos, esperamos que el Horizontal Pod Autoscaler (HPA) detecte la saturación en menos de 90 segundos y escale réplicas adicionales para mantener la latencia media por debajo de 800 ms».

Diseño de los Experimentos de Caos y Salvaguardas

Para evitar daños colaterales en el clúster, cada experimento fue parametrizado con un blast radius estrictamente limitado por selectores de etiquetas en el namespace otel-lab, duración acotada y mecanismos de reversión automática. La Tabla 1 sintetiza el diseño operacional.

Tabla 1

Matriz de diseño de experimentos de caos, blast radius y reversión automática

Experimento	Tipo de Fallo	Radio de Impacto (Blast Radius)	Duración	Rollback Automático
Exp. 1 (Red)	Network Latency (200 ms +/- 10 ms)	Solo pods app=service-b (100% réplicas) Namespace: otel-lab	5 min	Expiración TTL del CRD y limpieza de cola tc qdisc en el kernel
Exp. 2 (Pod Kill)	Pod Failure / Crash (Terminación dura)	Exactamente 1 pod de service-b (modo 'one') Preserva al menos 1 pod activo	Inmediato	Reconciliación automática vía ReplicaSet Controller de K8s
Exp. 3 (CPU)	Resource Exhaustion (2 workers al 75%)	1 pod de service-a (modo 'one') Namespace: otel-lab	3 min	Finalización de temporizador en worker y cgroups de Linux

Nota. Todos los experimentos fueron implementados mediante CRDs declarativos de Chaos Mesh en GKE.

Criterios de Aborto de Emergencia

Se estipuló detener inmediatamente cualquier prueba si la tasa de errores 5xx superaba el 5% sostenido durante 30 segundos, o si se detectaban reinicios anómalos en el plano de control o en PostgreSQL.

Marco de Observabilidad y Correlación Cross-Signal

La evaluación de las hipótesis se basó en los cuatro Golden Signals de SRE (Ewaschuk, 2016). La Tabla 2 presenta los indicadores de nivel de servicio (SLIs) instrumentados y las reglas de alerta configuradas en Prometheus.

Tabla 2

Indicadores de nivel de servicio (SLIs), consultas PromQL y alertas asociadas

SLI Objetivo	Métrica Telemetría	Expresión PromQL	Regla de Alerta
Latencia p99 (service-b)	otelcol_http_request_ duration_seconds_bucket	histogram_quantile(0.99, rate(otelcol_http_ request_duration_seconds_b ucket{job="service-	ChaosLatencyDetected (Umbral > 250 ms por 30s)

		b"}[1m]))	
Tasa de Errores (service-a)	otelcol_http_requests_total (status=5xx)	sum(rate(otelcol_http_requests_total{status=~"5.."}[1m])) / sum(rate(otelcol_http_requests_total[1m])) * 100	HighBurnRate_PAGE (Burn rate > 14.4x en 1h)
Saturación (service-a)	otelcol_http_active_requests	sum(otelcol_http_active_requests) by (endpoint)	AIOpsCorrelatedAnomaly (Saturación y p99 anómalo)

Nota. Consultas raspadas desde el OTel Collector expuesto en el puerto 8889 e integradas en Prometheus :9090.

Correlación de Trazas y Logs con TraceContext W3C

Cada petición genera un identificador trace_id hexadecimal de 32 dígitos que se inyecta en el cliente HTTPX mediante el encabezado traceparent y se adjunta a cada registro JSON de log. En Jaeger UI, una consulta a GET /order/{order_id} despliega claramente la jerarquía de spans: el span raíz HTTP en service-a, la consulta SQL fetch.order.db y la llamada remota call.service-b.inventory, permitiendo aislar visualmente si una degradación proviene de la base de datos o de la red.

Ejecución Empírica de los Experimentos en GKE y Evidencias

A diferencia de simulaciones teóricas, ambos experimentos fueron ejecutados directamente sobre el clúster de producción GKE utilizando el script setup.ps1 y manifiestos de Chaos Mesh.

Ejecución del Experimento 1: Network Latency (200 ms) en service-b

Se registraron 50 peticiones baseline con latencias entre 0.213 s y 0.515 s. A continuación, se inyectó el retardo de red con Chaos Mesh. El extracto de la salida de consola evidencia el comportamiento del sistema antes, durante y después del caos:

```
[11:09:59] Generando 50 peticiones baseline a service-a...
Req 1: OK (0.515s) | Req 2: OK (0.304s) | Req 3: OK (0.232s) ... Req 50: OK
(0.376s)
Baseline completado: Media = 0.282s | Error rate = 0.00%

[11:05:03] Aplicando NetworkChaos (200ms delay, 10ms jitter en service-b)...
networkchaos.chaos-mesh.org/latency-service-b-200ms created
Muestra Caos 1: Duracion = 4.3934s (Status: 200 OK)
[11:05:25] Alerta Prometheus: ChaosLatencyDetected pasa a FIRING (MTTD = 22
segundos)

[11:05:33] Revertiendo experimento (Rollback)...
```

```
networkchaos.chaos-mesh.org "latency-service-b-200ms" deleted
Post-Rollback: Duracion = 0.3382s (MTTR = 13 segundos)
```

Ejecución del Experimento 2: Pod Failure (Kill) en service-b

Para comprobar la alta disponibilidad N-1 y la capacidad del Kube-Proxy para reconmutar el tráfico sin intervención humana, se ejecutó un PodKill sobre una de las instancias activas de service-b mediante el comando .\scripts\setup.ps1 -Chaos2. El registro de la consola muestra la ejecución en vivo:

```
[13:19:46] INICIANDO EXPERIMENTO 2: Pod Failure en service-b (Alta
Disponibilidad N-1)
Pods de service-b antes del fallo:
NAME                         READY   STATUS    RESTARTS   AGE
service-b-56f78d79cb-tdj76   1/1     Running   0          167m
service-b-56f78d79cb-wsqj2   1/1     Running   0          167m

[13:19:46] Inyectando PodChaos (eliminacion no graceful de 1 replica)...
podchaos.chaos-mesh.org/pod-kill-service-b created

[13:19:50] Estado de los pods inmediatamente tras el fallo:
service-b-56f78d79cb-l46ht   1/1     Running   0          3s   <-- Pod
sustituto creado en 3s
service-b-56f78d79cb-tdj76   1/1     Running   0          168m

[13:19:51] Probando resiliencia transaccional de service-a durante el pod
kill:
Peticion durante Pod Failure 1: Status 200 OK (0.343s)
Peticion durante Pod Failure 2: Status 200 OK (0.433s)
Peticion durante Pod Failure 3: Status 200 OK (0.270s)
Resultado: El balanceador conmuto al pod superviviente con 0% de errores de
usuario.
```

Tabla 3

Comparación cuantitativa de resultados esperados vs. resultados reales obtenidos en GKE

Métrica / Escenario	Resultado Esperado	Resultado Real en GKE	Veredicto Técnico
Latencia Línea Base	< 0.450 s	0.2820 s (media de 50 peticiones)	Cumplido (Línea base sólida)
Latencia Exp. 1 (Red)	Degradación controlada < 5.0 s	4.3934 s (Retardo downstream retenido)	Cumplido (Hipótesis 1 validada)
Tasa de Errores Exp. 1	0.00% (Sin abortos)	0.00% (Transacciones 200 OK preservadas)	Cumplido (Resiliencia transaccional)
Disponibilidad Exp. 2 (Pod Kill)	Reconmutación N-1 sin fallos	100% éxito (0.343s, 0.433s, 0.270s)	Cumplido (Hipótesis 2 validada)

Tiempos Operacionales	MTTD < 120s | MTTR < 30s	MTTD = 22s | MTTR = 13s	Cumplido (Detección y reversión ágil)

Nota. Datos recolectados directamente desde el clúster otel-lab de GKE durante el Game Day.

Defectos Detectados Durante el Laboratorio y Correcciones Técnicas

Al igual que en la práctica anterior, la puesta en marcha de un laboratorio de ingeniería en la nube involucra desafíos reales de integración que deben documentarse con honestidad técnica. La Tabla 4 detalla los incidentes encontrados durante el despliegue y la solución adoptada para cada uno.

Tabla 4

Defectos de configuración y compatibilidad detectados, impacto y solución aplicada

Defecto Detectado	Impacto Técnico	Corrección Aplicada
Exporter jaeger deprecado en OTel Collector	El pod del Collector entraba en CrashLoopBackOff: 'exporters' unknown type: jaeger	Se migró la configuración a otlp/jaeger sobre gRPC (jaeger-svc:4317) con exporter debug.
jaeger-svc sin puerto 4317 expuesto	El Collector fallaba con Connection refused al intentar enviar spans a Jaeger	Se actualizó la definición del Service tipo LoadBalancer agregando el puerto otlp-grpc 4317.
Incompatibilidad de scripts Bash en Windows	PowerShell fallaba con 'chmod no se reconoce' y error por el operador & en background	Se desarrolló setup.ps1 con cmdlets nativos y Start-Process para ejecución en Windows.
Timeout HTTPX rígido de 5.0 s en service-a	Retención excesiva de conexiones durante el caos, amenazando con saturar workers de Uvicorn	Se formuló el plan de remediación con Circuit Breaker (pybreaker) y timeout agresivo de 1.0 s.

Nota. Registro de solución de incidentes de infraestructura resueltos durante la fase de puesta a punto en GKE.

Análisis Post-Mortem y Plan de Remediación SRE

Identificación de la Debilidad Sistémica

Aunque el Experimento 1 concluyó exitosamente en apariencia (no se produjeron errores 500 y las órdenes finalizaron), el análisis detallado de la telemetría reveló una vulnerabilidad crítica de diseño: riesgo inminente de agotamiento de trabajadores (Cascading Thread Starvation). La llamada saliente a service-b en service-a tiene fijado un timeout rígido de 5.0

segundos. Cuando service-b sufrió latencia de red, cada petición retuvo un socket y un hilo de ejecución durante 4.39 segundos. En un entorno con concurrencia real (> 100 req/s), el pool de trabajadores de Uvicorn se habría colapsado en pocos segundos, provocando que peticiones triviales o consultas que no requerían inventario quedaran bloqueadas sin responder.

Plan de Remediación

Para transformar el sistema en una arquitectura anti-frágil, se proponen tres acciones prioritarias:

1. Implementación de Circuit Breaker con Fallback Gracioso: Integrar la librería pybreaker en service-a. Si 5 llamadas consecutivas a service-b tardan más de 800 ms, el circuito se abre inmediatamente y retorna un valor de stock estimado en caché (Redis o memoria local), respondiendo al usuario en menos de 15 ms en modo degradado.

2. Reducción de Timeouts y Deadline Propagation: Reducir el timeout estático a 1.0 s y propagar la cabecera estándar Request-Timeout, para que si el orquestador aborta la espera, service-b no continúe gastando CPU inútilmente.

3. Auto-escalado por Saturación de Conexiones: Reconfigurar el HPA no solo por umbral de CPU, sino por la métrica custom otelcol_http_active_requests > 40, aprovisionando réplicas antes de que se agoten los workers.

Conclusiones

1. La observabilidad es el pilar habilitador del Chaos Engineering. Sin un pipeline de telemetría de alta resolución (OTel, Prometheus, Jaeger), inyectar fallos en producción o sandbox se reduce a una acción destructiva sin capacidad diagnóstica. Las métricas permitieron constatar que el sistema toleró el retardo funcionalmente, pero las trazas alertaron sobre la acumulación de latencia en la llamada downstream.

2. La ausencia de errores HTTP 500 no es sinónimo de resiliencia. El Experimento 1 demostró que un servicio puede continuar respondiendo 200 OK mientras sus conexiones se retienen peligrosamente durante más de 4 segundos, creando una trampa de saturación latente que colapsaría el sistema bajo carga concurrente.

3. La alta disponibilidad de Kubernetes debe complementarse a nivel de aplicación. Mientras que el Experimento 2 comprobó que el balanceador reconmutó eficazmente al pod superviviente ante la muerte de una instancia (N-1), la degradación por lentitud (Experimento 1) exige patrones de software como Circuit Breaker y Timeouts agresivos para evitar fallos en cascada.

Referencias

Basiri, A., Sengupta, N., Trzcinski, J., Harshan, K., Kalgutkar, N. y Hochstein, L. (2016). Chaos engineering. IEEE Software, 33(3), 35–41. https://doi.org/10.1109/MS.2016.64

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (Eds.). (2018). The site reliability workbook: Practical ways to implement SRE. O'Reilly Media.

Chaos Mesh Authors. (2024). Chaos Mesh documentation: Simulating faults in Kubernetes. Cloud Native Computing Foundation. https://chaos-mesh.org/docs/

Cloud Native Computing Foundation. (2024). OpenTelemetry specification: An observability framework for cloud-native software. https://opentelemetry.io/docs/specs/otel/

Ewaschuk, R. (2016). Monitoring distributed systems. En B. Beyer, C. Jones, J. Petoff y N. R. Murphy (Eds.), Site reliability engineering: How Google runs production systems (pp. 57–67). O'Reilly Media.

Google Cloud. (2024). Observability architecture: Designing telemetry pipelines with OpenTelemetry Collector. https://cloud.google.com/architecture/observability-opentelemetry

LitmusChaos Authors. (2024). LitmusChaos documentation: Cloud-native chaos engineering. CNCF Sandbox Project. https://docs.litmuschaos.io/

Ochoa Paipilla, M. F. (2026, 8 de octubre). Chaos engineering, métricas SRE y baseline dinámico de alertas [Sesión de clase]. Maestría en Arquitectura de Software, Universidad de La Sabana.

Sachto, A. y Walcer, A. (2022). Anatomy of an incident. Google Site Reliability Engineering. https://sre.google/resources/practices-and-processes/anatomy-of-an-incident/

World Wide Web Consortium. (2021). W3C Recommendation: Trace context (W3C standard for distributed context propagation). https://www.w3.org/TR/trace-context/